AWS

Database Services

Comprehensive Guide for Architects

Technical Deep Dive | 2026

Database Selection Framework

Workload Type

OLTP, OLAP, or Hybrid

Data Structure

Relational, Document, Key-Value, Graph

Scale Requirements

Read/Write throughput and storage needs

Performance

Latency and consistency requirements

Availability

RPO and RTO targets

Cost

TCO and operational overhead

Database Types Overview

Database Type	AWS Service	Best For
Relational (SQL)	RDS, Aurora	Traditional applications, transactions
Key-Value	DynamoDB	High-traffic web apps, gaming, IoT
In-Memory	ElastiCache	Caching, session stores, real-time analytics
Data Warehouse	Redshift	Business intelligence, analytics
Document	DocumentDB	Content management, catalogs, user profiles
Graph	Neptune	Social networks, fraud detection, knowledge graphs

Amazon RDS

Relational Database Service

Amazon RDS Overview

Managed relational database service that automates time-consuming administration tasks

Key Benefits

- Automated backups and patching
- Multi-AZ deployments for HA
- Read replicas for scaling
- Point-in-time recovery
- Monitoring and metrics

What's Managed

- OS patching and updates
- Database software installation
- Backup automation
- Failure detection
- Infrastructure provisioning

RDS Engine Options

MySQL

Most popular open-source database

Versions: 5.7, 8.0

PostgreSQL

Advanced open-source with rich features

Versions: 12, 13, 14, 15

MariaDB

MySQL-compatible with enhancements

Versions: 10.5, 10.6

Oracle

Enterprise-grade commercial database

BYOL or License Included

SQL Server

Microsoft's relational database

Express, Web, Standard, Enterprise

Db2

IBM's enterprise database

Standard & Advanced Editions

RDS Features and Benefits

Multi-AZ Deployments

Synchronous replication to standby in different AZ, automatic failover (1-2 minutes)

Read Replicas

Asynchronous replication for read scaling (up to 15), promote to standalone

Automated Backups

Daily snapshots with transaction logs, 1-35 days retention, point-in-time recovery

Encryption

At-rest with AWS KMS, in-transit with SSL/TLS, transparent encryption for Oracle/SQL Server

RDS Use Cases

Web and Mobile Applications

E-commerce platforms, content management systems, user authentication services

Enterprise Applications

ERP systems, CRM platforms, inventory management, financial systems

SaaS Applications

Multi-tenant applications requiring data isolation and compliance

Example Architecture

E-commerce Platform:

- Multi-AZ RDS MySQL for transactional data
- Read replicas for product catalog queries
- ElastiCache for session management
- Automated backups before major deployments

Amazon Aurora

Cloud-Native Relational Database

Amazon Aurora Overview

Cloud-optimized database engine providing **5x throughput of MySQL** and **3x throughput of PostgreSQL**

Key Advantages

- Up to 128 TB storage auto-scaling
- 15 read replicas with sub-10ms latency
- 6-way replication across 3 AZs
- Continuous backup to S3
- Fast database cloning
- Global database spanning regions

Compatible Engines

Aurora MySQL

Compatible with MySQL 5.7 and 8.0

Aurora PostgreSQL

Compatible with PostgreSQL 12, 13, 14, 15

Aurora Performance and Architecture

Storage Architecture

Distributed, fault-tolerant, self-healing storage that automatically scales in 10GB increments up to 128TB

High Availability

6 copies across 3 AZs, survives loss of 2 copies for writes and 3 copies for reads, automatic failover under 30 seconds

Performance Features

Parallel query processing, query cache, optimized for cloud infrastructure, lower storage I/O costs

Aurora Serverless

Auto-scaling with per-second billing, ideal for variable workloads and dev environments

Global Database

Single database spanning regions, sub-second replication, DR with RPO of 1 second

Aurora Use Cases

Enterprise Apps

Mission-critical applications requiring high availability and performance at scale

Example: Financial trading platform with millisecond latency requirements

SaaS Platforms

Multi-tenant SaaS with variable workloads and global user base

Example: CRM platform serving customers across multiple continents

Gaming

Real-time leaderboards, player data, and matchmaking systems

Example: Multiplayer game with millions of concurrent users

RDS vs Aurora Comparison

Feature	Amazon RDS	Amazon Aurora
Performance	Standard database performance	5x MySQL, 3x PostgreSQL performance
Storage	Up to 64 TB (engine dependent)	Up to 128 TB, auto-scaling
Read Replicas	Up to 5 (MySQL), 5 (PostgreSQL)	Up to 15 with lower lag
Replication	Asynchronous to replicas	6-way across 3 AZs
Failover Time	1-2 minutes	Under 30 seconds
Cost	Lower entry point	~20% more expensive, better value at scale

Amazon DynamoDB

Serverless NoSQL Database

Amazon DynamoDB Overview

Fully managed, serverless NoSQL database delivering single-digit millisecond performance at any scale

Key Characteristics

- Serverless with auto-scaling
- Single-digit millisecond latency
- Virtually unlimited throughput
- Built-in security and backup
- Global tables for multi-region
- ACID transactions support

Performance

10

Trillion requests per day

20M

Peak requests per second

DynamoDB Core Concepts

Tables and Items

Schema-less tables with items and attributes, no schema required

Primary Keys

Partition key or composite key for unique ID and queries

Indexes

GSI and LSI for flexible query patterns beyond primary key

Capacity Modes

On-demand (pay per request) or Provisioned (RCU/WCU)

Streams

Capture item-level changes in near real-time for triggers, replication, or analytics

DynamoDB Features

Global Tables

Multi-region, fully replicated tables with active-active configuration

Sub-second replication between regions

Point-in-Time Recovery

Continuous backups with restore to any point in last 35 days

No performance impact

DAX

DynamoDB Accelerator in-memory cache

Microsecond latency for read-heavy workloads

Transactions

ACID transactions across multiple items and tables

All-or-nothing operations

TTL

Time-to-live for automatic item expiration

No additional cost

Encryption

Encryption at rest by default using AWS KMS

In-transit encryption with TLS

DynamoDB Use Cases

Mobile and Gaming

User profiles, session data, leaderboards, game state management with millions of users

IoT Applications

Time-series data from sensors, device metadata, event tracking at massive scale

Real-Time Bidding

Ad tech platforms requiring microsecond response times and unpredictable traffic spikes

Example: Gaming Platform

- **Player profiles:** On-demand capacity mode
- **Leaderboards:** GSI for score queries
- **Session data:** TTL for automatic cleanup
- **Game events:** DynamoDB Streams → Lambda
- **Global players:** Global tables across regions

Amazon ElastiCache

In-Memory Data Store

Amazon ElastiCache Overview

Fully managed in-memory caching service supporting Redis and Memcached for microsecond latency

Key Benefits

- Sub-millisecond response times
- Reduce database load by 80%+
- Fully managed with auto-discovery
- Automatic failover and backup
- Scaling with minimal downtime
- Enhanced security with VPC

Common Use Cases

- Database query result caching
- Session store for web apps
- Real-time analytics
- Leaderboards and counting
- Message queues and pub/sub
- Rate limiting and throttling

Redis vs Memcached

Feature	ElastiCache for Redis	ElastiCache for Memcached
Data Types	Strings, Lists, Sets, Sorted Sets, Hashes, Bitmaps, HyperLogLogs	Simple key-value strings only
Persistence	Snapshots and AOF (Append-Only File)	No persistence
Replication	Multi-AZ with automatic failover, up to 5 read replicas	No replication
Transactions	Supports transactions and Lua scripting	No transactions
Pub/Sub	Yes, with pattern matching	No
Multi-threading	Single-threaded	Multi-threaded, better for large nodes
Best For	Complex data structures, persistence, HA	Simple caching, horizontal scaling

ElastiCache Use Cases

Database Caching

Cache frequently accessed data from RDS or DynamoDB

Pattern: Cache-aside, write-through, or lazy loading

Session Management

Store user sessions across stateless application servers

Benefit: Fast access, automatic expiration with TTL

Real-Time Analytics

Leaderboards, counters, rate limiting, streaming analytics

Use Redis: Sorted sets for rankings, atomic operations

Amazon Redshift

Data Warehouse

Amazon Redshift Overview

Fully managed petabyte-scale data warehouse optimized for complex analytics and BI workloads

Key Capabilities

- Columnar storage for analytics
- Massively parallel processing (MPP)
- Standard SQL and BI tool support
- Automated backups to S3
- Concurrency scaling for bursts
- Data lake integration via Spectrum

Performance

Up to 10x faster than traditional data warehouses

3x better price-performance than other cloud data warehouses

Scales to exabytes with Redshift Spectrum

Redshift Architecture

Cluster Architecture

Leader node coordinates queries, compute nodes execute in parallel, scales 1 to 128 nodes

Columnar Storage

Data stored by column for efficient compression, only reads needed columns

Redshift Spectrum

Query exabytes in S3 without loading, extends queries beyond cluster storage

Concurrency Scaling

Auto-adds capacity for spiky workloads, first hour free daily

Node Types

RA3: Managed storage, compute-storage separation | **DC2:** Dense compute | **DS2:** Dense storage (legacy)

Redshift Use Cases

Business Intelligence

Complex queries across large datasets, dashboards, reports, data visualization with Tableau, Power BI, QuickSight

Data Lake Analytics

Query structured and semi-structured data in S3 using Spectrum, unite data warehouse and lake

Operational Analytics

Near real-time analytics on operational data, combine with streaming via Kinesis

Example: Retail Analytics

- **Sales data:** Multi-year transaction history in Redshift
- **Log data:** Raw clickstream in S3 via Spectrum
- **Real-time:** Kinesis → Redshift streaming
- **Reporting:** QuickSight dashboards
- **ML:** SageMaker integration for predictions

Specialized Databases

DocumentDB & Neptune

Amazon DocumentDB

MongoDB-compatible document database designed for JSON workloads with AWS scale and reliability

Key Features

- MongoDB 3.6, 4.0, 5.0 compatibility
- Fully managed with auto-scaling
- Up to 15 read replicas
- Continuous backup to S3
- 6-way replication across 3 AZs
- Encryption at rest and in transit

Use Cases

- Content management systems
- Product catalogs
- User profiles and preferences
- Mobile app backends
- Real-time big data

Why not MongoDB on EC2? Fully managed, better HA, automated backups, separation of compute/storage

Amazon Neptune

Fully managed graph database supporting Apache TinkerPop Gremlin and W3C SPARQL query languages

Key Features

- Supports property graph and RDF
- ACID transactions
- Up to 15 read replicas
- Point-in-time recovery
- Multi-AZ high availability
- Fast failover (< 30 seconds)

Use Cases

- Social networking graphs
- Fraud detection patterns
- Recommendation engines
- Knowledge graphs
- Network and IT operations
- Life sciences research

Graph queries traverse relationships orders of magnitude faster than relational JOINs

Database Comparison Matrix

Service	Type	Latency	Scale	Best For
RDS	Relational (SQL)	Single-digit ms	Vertical, up to 64TB	Traditional apps, ACID transactions
Aurora	Relational (SQL)	Single-digit ms	Up to 128TB, 15 replicas	High performance, cloud-native apps
DynamoDB	Key-Value/Document	Single-digit ms	Virtually unlimited	High-scale web apps, gaming, IoT
ElastiCache	In-Memory	Sub-millisecond	Horizontal scaling	Caching, session store, real-time
Redshift	Data Warehouse	Seconds	Petabytes	Analytics, BI, complex queries
DocumentDB	Document	Single-digit ms	Up to 64TB	Content management, catalogs
Neptune	Graph	Single-digit ms	Billions of relationships	Social networks, fraud detection

When to Use Each Database

RDS

ACID transactions, JOINs, SQL apps

Aurora

High performance SQL, global apps

DynamoDB

Massive scale, serverless NoSQL

ElastiCache

Caching, sub-millisecond latency

Redshift

OLAP, analytics, BI reporting

DocumentDB

MongoDB compatible, JSON docs

Migration Strategies

Moving Databases to AWS

Migration Strategies Overview

Rehost (Lift-and-Shift)

Move database as-is to AWS

Tools: AWS Application Migration Service, database backup/restore

Speed: Fast, low risk

Replatform

Move to managed service with minimal changes

Tools: AWS DMS, native replication

Speed: Moderate, better TCO

Refactor

Redesign for cloud-native architecture

Tools: AWS DMS, Schema Conversion Tool, application rewrites

Speed: Slow, maximum benefit

AWS Database Migration Service

Migrate databases to AWS with minimal downtime using continuous replication

Key Capabilities

- Homogeneous and heterogeneous migrations
- Source database remains operational
- One-time or ongoing replication
- Automatic schema conversion
- Multi-AZ for high availability

Migration Patterns

Homogeneous

Same engine (Oracle → RDS Oracle)

Heterogeneous

Different engines (Oracle → PostgreSQL)

Development/Test

Replicate production to lower environments

Migration Best Practices

1. Assessment Phase

Inventory databases, assess compatibility, estimate costs, identify dependencies

2. Proof of Concept

Test migration with representative workload, validate performance, verify functionality

3. Migration Execution

Use DMS for continuous replication, minimize downtime with cutover window, validate data integrity

4. Optimization

Right-size instances, enable auto-scaling, implement caching strategies, optimize queries for cloud

5. Monitoring and Operations

Set up CloudWatch alarms, enable Enhanced Monitoring, implement backup strategies, establish runbooks

Cost Optimization Tips

Right-Sizing

- Monitor CloudWatch
- Use Performance Insights
- Start small, scale up
- Burstable T3/T4g

Reserved Instances

- 1 or 3-year commits
- Save up to 69%
- Convertible option
- Use Savings Plans

Storage

- Aurora auto-scales
- Reduce backup retention
- Right IOPS tier
- Archive to S3

Read Scaling

- Use read replicas
- Add ElastiCache
- Aurora Serverless
- Offload to Redshift

DynamoDB

- On-demand for variable
- Provisioned for steady
- Reserved capacity
- TTL auto-delete

Monitoring

- Billing alarms
- Cost Explorer
- Tag resources
- Monthly reviews

Summary

Key Takeaways

- **Choose the right tool:** Match database type to workload requirements
- **Start with managed services:** Reduce operational overhead
- **Design for high availability:** Multi-AZ, read replicas, backups
- **Plan your migration:** Assess, pilot, execute, optimize
- **Optimize continuously:** Right-size, cache, monitor costs

Quick Decision Guide

Transactions + SQL? → RDS or Aurora

Massive scale + NoSQL? → DynamoDB

Caching? → ElastiCache

Analytics? → Redshift

Documents? → DocumentDB

Relationships? → Neptune

Next Steps

- Assess your current database workloads
- Review AWS documentation and best practices
- Set up a proof-of-concept environment
- Engage with AWS Solutions Architects
- Plan your migration roadmap

Questions?